

第 4 章 函数和递归

学习目标

- ☑ 掌握多参数、单返回值的数学函数的定义和使用方法
- ☑ 学会用 `typedef` 定义结构体
- ☑ 理解函数调用时用实参给形参赋值的过程
- ☑ 学会定义局部变量和全局变量
- ☑ 理解调用栈和栈帧，学会用 `gdb` 查看调用栈并选择栈帧
- ☑ 理解地址和指针
- ☑ 理解递归定义和递归函数
- ☑ 理解可执行文件中的正文段、数据段和 BSS 段
- ☑ 熟悉堆栈段，了解栈溢出的常见原因

运用前 3 章的知识尽管在理论上已经足以写出所有算法程序了，但实际上稍微复杂一点的程序往往由多个函数组成。函数是“过程程序设计”的自然产物，但也产生了局部变量、参数传递方式、递归等诸多新的知识点。本章的主要目的在于理解这纷繁复杂的、最后的语法。同时，通过 `gdb`，可以从根本上帮助读者理解，看清事物的本质。最后，通过一些实际的竞赛题目帮助读者学习编写算法程序的一般方法和技巧。

4.1 自定义函数和结构体

我们已经用过了许多数学函数，如 `cos`、`sqrt` 等。能不能自己写一个呢？没问题。下面就编写一个计算两点欧几里德距离的函数：

```
double dist(double x1, double y1, double x2, double y2)
{
    return sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2));
}
```

提示 4-1：C 语言中的数学函数可以定义成“返回类型 函数名(参数列表) { 函数体 }”，其中函数体的最后一条语句应该是“`return 表达式;`”。

这里，参数和返回值的类型一般是前面介绍过的“一等公民”，如 `int` 或者 `double`，也可以是 `char`。可不可以是数组呢？也不是不可以，但是比较麻烦，稍后再考虑。有时，函数并不需要返回任何值，例如，它只是用 `printf` 向屏幕输出一些内容。这时只需定义函数返回类型为 `void`，并且无须使用 `return`（除非希望在函数运行中退出函数）。

提示 4-2：函数的参数和返回值最好是“一等公民”，如 `int`、`char` 或者 `double` 等。其他“非一等公民”作为参数和返回值要复杂一些。如果函数不需要返回值，则返回类型应写成 `void`。

注意这里的 `return` 是一个动作，而不是描述。

提示 4-3：如果在执行函数的过程中碰到了 `return` 语句，将直接退出这个函数，不去执行后面的语句。相反，如果在执行过程中始终没有 `return` 语句，则会返回一个不确定的值。幸好，`-Wall` 可以捕捉到这一可疑情况并产生警告。

顺便说一句，`main` 函数也是有返回值的！到目前为止，我们总是让它返回 0，这个 0 是什么意思呢？尽管没有专门说明，读者应该已经发现了，`main` 函数是整个程序的入口。换句话说，有一个“其他的程序”来调用这个 `main` 函数——如操作系统、IDE、调试器，甚至自动评测系统。这个 0 代表“正常结束”，即返回给调用者。在算法竞赛中，除了有特殊规定之外，请总是让其返回 0，以免评测系统错误地认为程序异常退出了。

提示 4-4：在算法竞赛中，请总是让 `main` 函数返回 0。

函数不一定要一步得出结果。下面是上述函数的另一种写法：

```
double dist(double x1, double y1, double x2, double y2)
{
    double dx = x1-x2;
    double dy = y1-y2;
    return hypot(dx, dy);
}
```

这里用到了一个新的数学函数——`hypot`，相信读者能猜到它的意思^①。这个例子也说明，一个函数也可以调用其他函数——在自定义函数中写代码和在 `main` 函数中写代码并没有什么区别，以前讲过的知识都适用。

下面来思考一个问题：这个函数是否好用？通常，`x1` 和 `y1` 在语义上属于一个整体 (`x1,y1`)，而 `x2` 和 `y2` 属于另一个整体 (`x2,y2`)，代表两个点的坐标。那么能否设计一个函数，其参数是明显的两个点，而不是 4 个 `double` 型的坐标值呢？

```
struct Point{ double x, y; };

double dist(struct Point a, struct Point b)
{
    return hypot(a.x-b.x, a.y-b.y);
}
```

这里出现了一个新内容。上述代码中定义了一个称为 `Point` 的结构体，包含两个域：`double` 型的 `x` 和 `y`。

提示 4-5：在 C 语言中，定义结构体的方法为“`struct 结构体名称{ 域定义 };`”，注意花括号的后面还有一个分号。

这样用起来有些不合习惯：所有用到 `Point` 的地方都得写一个 `struct`。有一个方法可以

^① 注意：这个函数不是 ANSI C 的。

避开这些 struct，让结构体用起来和 int、double 这样的“原生”类型更接近：

```
typedef struct{ double x, y; }Point;

double dist(Point a, Point b)
{
    return hypot(a.x-b.x, a.y-b.y);
}
```

代码中虽然没少几个字符，但是看上去清爽多了！

提示 4-6：为了方便，往往用“typedef struct { 域定义; } 类型名;”的方式定义一个新类型名。这样，就可以像原生数据类型一样使用这个自定义类型。

计算组合数。编写函数，参数是两个非负整数 n 和 m ，返回组合数 $C_n^m = \frac{n!}{m!(n-m)!}$ ，

其中 $m \leq n \leq 25$ 。例如， $n=25$ ， $m=12$ 时答案为 5200300。

【分析】

既然题目中的公式多次出现 $n!$ ，将其作为一个函数编写是比较合理的：

程序 4-1 组合数（有问题）

```
long long factorial(int n){
    long long m = 1;
    for(int i = 1; i <= n; i++){
        m *= i;
    }
    return m;
}

long long C(int n, int m)
{
    return factorial(n) / (factorial(m) * factorial(n-m));
}
```

由此可见，编写函数并不困难。写完之后的函数可以像 cos、sqrt 等库函数一样被调用。

“别忘了测试！”如果你这样说，请为自己鼓掌。还记得第 2 章那个“阶乘”之和的第一个程序吗？那个程序溢出了。那这个程序呢？很不幸： $n=21$ ， $m=1$ 的返回值竟然是 -1。手算不难得到： $n=21$ ， $m=1$ 的正确结果是 21，显然结果不符。

提示 4-7：即使最终答案在所选择的数据类型范围之内，计算的中间结果仍然可能溢出。

这个题目还说明：即使认为题目在“暗示”你使用某种语言特性，也应该深入分析，不能贸然行事。如何避免中间结果溢出？办法是进行“约分”。一个简单的方法是利用 $n!/m! = (m+1)(m+2) \cdots (n-1)n$ 。虽然不能完全避免中间结果溢出，但是对于题目给出的范围已经可以保证得到正确的结果了。代码如下：

程序 4-2 组合数

```
long long C(int n, int m) {
    if(m < n-m) m = n-m;
    long long ans = 1;
    for(int i = m+1; i <= n; i++) ans *= i;
    for(int i = 1; i <= n-m; i++) ans /= i;
    return ans;
}
```

上述代码还有一个小技巧：当 $m < n-m$ 时把 m 变成 $n-m$ 。请读者思考这样做的意图。另外，这个函数里笔者改变了参数 m 的值。这样做并不会影响到函数的调用者，具体原因会在 4.2 节详细讨论。

提示 4-8：对复杂的表达式进行化简有时不仅能减少计算量，还能减少甚至避免中间结果溢出。

素数判定。编写函数，参数是一个正整数 n ，如果它是素数，返回 1，否则返回 0。

【分析】

根据定义，被 1 和它自身整除的、大于 1 的整数称为素数。这种“判断一个事物是否具有某一性质”的函数还有一个学术名称——谓词（predicate），下面程序中将写一个谓词。

程序 4-3 素数判定（有问题）

```
//n=1 或者 n 太大时请勿调用
int is_prime(int n)
{
    for(int i = 2; i*i <= n; i++)
        if(n % i == 0) return 0;
    return 1;
}
```

注意这里用到了两个小技巧。一是只判断不超过 $\sqrt{x}$ 的整数 i （想一想，为什么）。二是及时退出：一旦发现 x 有一个大于 1 的因子，立刻返回 0（假），只有最后才返回 1（真）。函数名的选取是有章可循的，“is_prime”取自英文“is it a prime?”（它是素数吗？）。

提示 4-9：建议把谓词（用来判断某事物是否具有某种特性的函数）命名成“is_xxx”的形式，返回 int 值，非 0 表示真，0 表示假。

注意程序 4-2 中 is_prime 函数上方的注释：不要用在 $n=1$ 或者 n 太大时调用。这是为什么呢？ n 太小时不难解释： $n=1$ 会被错误地判断为素数（因为确实没有其他因子）。 n 太大时的理由则不明显： $i*i$ 可能会溢出！如果 n 是一个接近 int 的最大值的素数，则当循环到 $i=46340$ 时， $i*i=2147395600 < n$ ；但 $i=46341$ 时， $i*i=2147488281$ ，超过了 int 的最大值，溢出变成负数，仍然满足 $i*i < n$ 。若 n 不是太大，可能出现 101128442 溢出后等于 2147483280，终止循环；但如果 $n=2147483647$ ，循环将一直进行下去。